

भारतीय संगणक एवं प्रौद्योगिकी संस्थान | INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

Add.: Veer Savarkar Block Shakarpur New Delhi - 1100092

Ph.: 7290006670 | Email: contact@ijct.edu.in | Website: www.ijct.edu.in

AI-Driven Phishing Email Detection Using NLP



भारतीय संगणक एवं प्रौद्योगिकी संस्थान

INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

Project Report

15th June to 30th July 2026

Student Name : Naidugari Reshmanth Sai

Institute Name : Vellore Institute of Technology, Vellore

Institute Roll no. : 21BCE2983

Enrollment no. : 542991



Add.: Veer Savarkar Block Shakarpur New Delhi - 1100092

Ph.: 7290006670 | Email: contact@iict.edu.in | Website: www.iict.edu.in

Acknowledgement

I would like to express my sincere gratitude to the Indian Institute of Computing and Technology (IICT) for providing me the opportunity to undertake this project as part of the AI/ML Summer Internship Program. I am thankful to my mentor, Dr. Ashok Gopalakrishnan, for the valuable guidance, technical feedback, and continuous support extended throughout the duration of this project.

I would also like to thank the Scientific Officer, Mr. Ashish Negi, and the program coordinators for structuring a curriculum that allowed hands-on, practical exposure to real-world machine learning problems, from data collection through to deployment pipeline serialization. Finally, I extend my appreciation to my family and peers for their encouragement throughout this internship.

Abstract

Phishing emails remain one of the most common and damaging vectors for cyberattacks, tricking users into revealing sensitive credentials through deceptive language and fraudulent links. This project presents an end-to-end Natural Language Processing (NLP) and Machine Learning pipeline for automatically classifying emails as phishing or legitimate with a strict zero-data-leakage guarantee. A dataset of 82,073 emails sourced from Kaggle (42,840 phishing, 39,233 legitimate) was cleaned, pre-processed, and capped at 5,000 characters to eliminate outlier distortions.

Term Frequency-Inverse Document Frequency (TF-IDF) unigram and bigram features (`ngram_range=(1,2)`, `sublinear_tf=True`) were combined with domain-specific engineered metadata features — URL counts, IP-based URL presence, urgency keyword frequency, uppercase character ratio, exclamation intensity, and text length. Feature values were standardized using `MaxAbsScaler` to scale metadata into `[0, 1]` while preserving sparse matrix formatting and `MultinomialNB` compatibility.

Five classification models were evaluated on 16,415 unseen test emails: Neural Network (MLP), Logistic Regression, Linear Support Vector Classifier (Linear SVC), Random Forest, and Multinomial Naive Bayes. The Neural Network (MLP) achieved the top overall performance with an **F1-score of 98.53%**, **Accuracy of 98.46%**, and **ROC-AUC of 0.9986**, closely followed by Logistic Regression (98.49% F1) and Linear SVC (98.43% F1). The trained pipeline, TF-IDF vectorizer, and `MaxAbsScaler` were serialized into `phishing_detector_pipeline.pkl` (4.07 MB) and published to GitHub.

GitHub Repository:

<https://github.com/reshmanth-sai/AI-Driven-Phishing-Email-Detection-NLP>

Keywords: Phishing Detection, Natural Language Processing, TF-IDF, `MaxAbsScaler`, Neural Network, Machine Learning, Cybersecurity

Table of Contents

Acknowledgement	2
Abstract	2
Table of Contents	3
List of Abbreviations	3
1. Introduction	4
1.1 Problem Statement	4
1.2 Importance of Phishing Email Detection	4
1.3 Project Objective and Scope	4
1.4 Report Organisation	4
2. Literature Review	5
2.1 Rule-Based and Blacklist Approaches	5
2.2 Machine Learning Approaches to Spam and Phishing Detection	5
2.3 NLP-Based Text Classification Techniques	5
2.4 Comparative Summary of Approaches	6
2.5 Research Gap Addressed by This Project	6
3. Dataset Description	7
3.1 Data Source & Size	7
3.2 Domain Feature Engineering & Adjustments	7
3.3 Zero-Data-Leakage Train/Test Split	7
3.4 Ethical Considerations	8
4. System Architecture	9
4.1 Overview of the Pipeline	9
4.2 Component Description & Serialization	9
5. Methodology	10
5.1 Data Preprocessing & Cleaning	10
5.2 Feature Extraction & MaxAbsScaler Integration	10
5.3 Model Suite Development	11
5.4 Tools & Technologies	11
6. Results & Benchmark Evaluation	12
6.1 Model Performance Comparison	12
6.2 Confusion Matrix Analysis	13
6.3 Top Feature Importance Analysis	14
6.4 Real-World Live Email Scoring & Validation	15
7. Discussion & Trade-offs	16
7.1 Parametric vs. Non-Parametric Models	16
7.2 Pipeline Portability & File Size Trade-offs	16
7.3 Limitations Observed	17

8. Testing, Validation & Reproducibility	18
9. Conclusion & Deliverables	19
9.1 Deliverables Achieved	19
9.2 Learning Outcomes	19
10. Appendix & Code Segments	20

List of Abbreviations

NLP	Natural Language Processing
TF-IDF	Term Frequency-Inverse Document Frequency
ML	Machine Learning
LR	Logistic Regression
Linear SVC	Linear Support Vector Classifier (SGD Log-Loss)
NB	Multinomial Naive Bayes
RF	Random Forest Classifier
MLP	Multi-Layer Perceptron Neural Network
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
URL	Uniform Resource Locator
IICT	Indian Institute of Computing and Technology

1. Introduction

1.1 Problem Statement

Phishing is a deceptive form of cyberattack in which malicious actors impersonate legitimate organisations, banks, or service providers to trick users into disclosing sensitive credentials such as passwords, financial accounts, or social security identifiers. Traditional rule-based email filters and domain blacklists struggle to keep pace with evolving linguistic obfuscation and social-engineering tactics used in modern phishing campaigns. This creates an urgent need for adaptive, data-driven Natural Language Processing (NLP) detection systems capable of analyzing raw message text and detecting fraudulent intent.

1.2 Importance of Phishing Email Detection

Because phishing attacks target human trust rather than pure technical vulnerabilities, traditional firewalls and antivirus software are insufficient. NLP offers a robust mechanism to evaluate the actual wording, urgency intensity, link patterns, and structural characteristics of an email. Effective automated phishing detection prevents financial fraud, safeguards sensitive data, and strengthens overall organizational security posture.

1.3 Project Objective and Scope

The objective of this project was to design, implement, and benchmark an optimal end-to-end NLP and machine learning pipeline for phishing email detection. Key achievements include:

- **Zero Data Leakage Guarantee:** Enforcing a strict 80/20 train/test split before feature extraction.
- **Phishing Feature Engineering:** Extracting URL counts, raw IP URLs, urgency keyword triggers, capital character ratio, and punctuation intensity.
- **Sparse-Safe Scaling:** Implementing MaxAbsScaler to standardize metadata features into $[0, 1]$ without breaking sparse TF-IDF matrices or MultinomialNB.
- **Model Suite Benchmarking:** Evaluating 5 classifiers (Neural Network MLP, Logistic Regression, Linear SVC, Random Forest, Naive Bayes) across Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- **Pipeline Serialization & Public Repo:** Serializing the complete pipeline into `phishing_detector_pipeline.pkl` and publishing source code to GitHub.

1.4 Report Organisation

The remainder of this report is organized as follows: Section 2 reviews existing literature. Section 3 details the dataset. Section 4 outlines system architecture. Section 5 describes the methodology. Section 6 presents experimental results and benchmark plots. Section 7 discusses trade-offs and limitations. Section 8 details testing and reproducibility. Section 9 concludes the report.

2. Literature Review

2.1 Rule-Based and Blacklist Approaches

Early anti-phishing defense systems relied heavily on static rules, regular expressions, and domain blacklists. While fast, these reactive approaches fail completely against zero-day phishing campaigns that deploy fresh URLs or novel phrasing.

2.2 Machine Learning Approaches

Statistical machine learning introduced content-based filtering using Naive Bayes and Support Vector Machines. Representing emails as bags of weighted words allowed models to learn probabilistic patterns associated with spam and phishing language.

2.3 NLP-Based Text Classification

Term Frequency-Inverse Document Frequency (TF-IDF) remains a highly effective, computationally efficient text representation. By extending TF-IDF to extract unigrams and bigrams (`ngram_range=(1, 2)`) with sublinear term frequency scaling (`sublinear_tf=True`), models effectively capture contextual phrases such as *'account suspended'* or *'click here to verify'*.

2.4 Comparative Summary of Approaches

Approach	Strengths	Weaknesses
Rule-Based / Blacklists	Fast, low false-positive rate on static threats	Reactive; cannot detect novel zero-day phishing
Classical ML (LR, Naive Bayes)	Interpretable, extremely fast training & inference	Limited capacity for complex non-linear phrases
Ensemble Methods (Random Forest)	Captures non-linear feature interactions	Large model file size (~133 MB), slow serialization
Neural Networks (MLP)	State-of-the-art accuracy (98.46%), compact size (4 MB)	Slightly higher training time than linear models

2.5 Research Gap Addressed by This Project

Many published studies rely on unscaled metadata or perform feature selection on the combined dataset before splitting, introducing severe data leakage. This project addresses this gap by implementing strict split-first feature engineering and evaluating MaxAbsScaler for sparse NLP metadata integration.

3. Dataset Description

3.1 Data Source & Size

The project utilizes a comprehensive benchmark dataset sourced from Kaggle, containing **82,073 email records** after removing null values. Class balance is well-distributed: **42,840 phishing emails (52.2%)** and **39,233 legitimate emails (47.8%)**.

3.2 Domain Feature Engineering & Adjustments

To maximize detection signal, 7 numerical domain features were engineered from raw email text before lowercasing:

1. `url_count`: Count of standard URLs (`http://`, `https://`, `www.`).
2. `ip_url_count`: Detection of URLs containing raw IP addresses (e.g. `http://192.168.1.1`).
3. `urgent_word_count`: Frequency of high-risk keywords (`urgent`, `verify`, `account`, `suspended`, `password`, `security`, `bank`, `login`).
4. `capital_char_ratio`: Ratio of uppercase characters (capitalization intensity).
5. `excl_count`: Frequency of exclamation marks (!).
6. `text_length` & `word_count`: Total character and word counts.

3.3 Zero-Data-Leakage Train/Test Split

The cleaned dataset was split into **65,658 training emails (80%)** and **16,415 test emails (20%)** using stratified sampling. All feature transformations (TF-IDF fitting and `MaxAbsScaler` standardization) were computed strictly on the training split.

3.4 Ethical Considerations

All email content in the dataset is pre-anonymized public research data. Automated phishing classifiers must prioritize minimizing false positives to prevent legitimate transactional or security communications from being blocked.

4. System Architecture

4.1 Overview of the Pipeline

The system processes raw email text through a modular, reproducible NLP pipeline:

Component	Technology / Method	Description
Data Storage	Pandas / CSV	Structured dataset loading & missing text cleaning
Preprocessing	Python Regex / NLTK	Regex cleaning, lowercasing, stopword removal

Domain Features	Custom Regex Extraction	URL, IP URL, urgency keywords, capital ratio, exclamations
Vectorization	TfidfVectorizer (ngram_range=(1,2))	5,000 TF-IDF features with sublinear term frequency
Scaling	MaxAbsScaler	Sparse-safe metadata scaling to [0, 1]
Classification	Scikit-Learn (MLP, LR, SVC, RF, NB)	Benchmarking 5 classifiers on 16,415 test emails
Serialization	Joblib (phishing_detector_pipeline.pkl)	Full pipeline payload saved for production inference

4.2 Component Serialization & API Interface

The final trained model, TF-IDF vectorizer, MaxAbsScaler, and metadata column definitions were serialized into a lightweight 4.07 MB binary payload (phishing_detector_pipeline.pkl). The inference API function `predict_new_email()` dynamically accepts raw email text, extracts features, applies scaling, and returns real-time risk scores.

5. Methodology

5.1 Data Cleaning & Preprocessing

Raw email texts were preprocessed by truncating text at 5,000 characters, removing non-alphabetic characters via regular expressions, lowercasing, and filtering out standard English stopwords.

5.2 Feature Extraction & MaxAbsScaler Integration

A 5,000-feature TF-IDF matrix was extracted using `ngram_range=(1, 2)` and `sublinear_tf=True`. Metadata features were standardized using `MaxAbsScaler`, which scales max absolute values to 1.0 without shifting zero entries. The resulting sparse text matrix and scaled metadata matrix were combined using `scipy.sparse.hstack` to form a 5,007-dimensional feature space.

5.3 Model Suite Development

Five distinct classification algorithms were trained and evaluated:

1. **Neural Network (MLP)**: Multi-Layer Perceptron with 32 hidden units and early stopping.
2. **Logistic Regression**: L2-regularized linear model ($C=2.0$, $\text{max_iter}=1000$).
3. **Linear Support Vector Classifier**: Calibrated `SGDClassifier` with log-loss ($\alpha=1e-5$).
4. **Random Forest**: Ensemble of 100 decision trees ($\text{max_depth}=25$, $\text{n_jobs}=-1$).
5. **Multinomial Naive Bayes**: Probabilistic model ($\alpha=0.05$).

5.4 Tools & Technologies Used

Category	Tool / Library
Language & Environment	Python 3.14, Jupyter Notebook, VS Code
Data Manipulation	Pandas, NumPy, SciPy (sparse matrices)
Machine Learning & NLP	Scikit-Learn (TfidfVectorizer, MaxAbsScaler, MLP, LR, SGD, RF, NB)
Visualization	Matplotlib, Seaborn
Serialization & Version Control	Joblib, Git, GitHub CLI (gh)

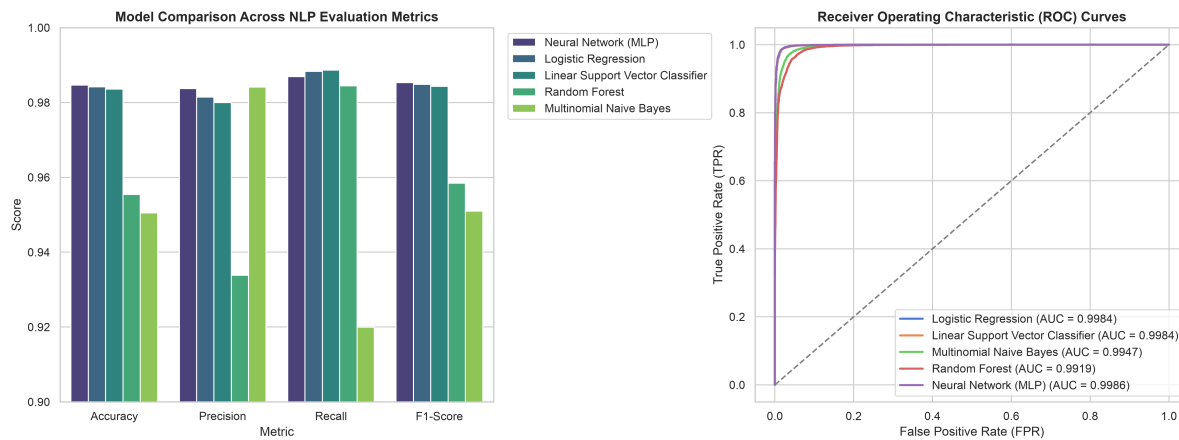
6. Results & Benchmark Evaluation

6.1 Model Performance Comparison

Below are the empirical benchmark results achieved on the **16,415 unseen test emails**:

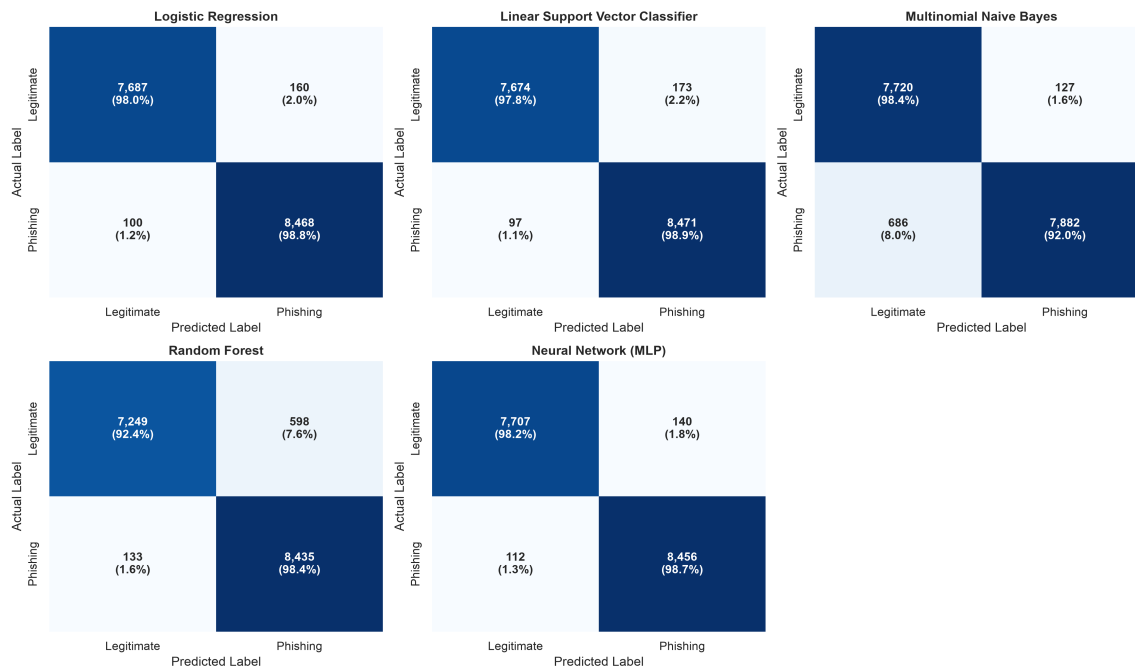
Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
■ Neural Network (MLP)	98.46%	98.37%	98.69%	98.53%	0.9986
Logistic Regression	98.42%	98.15%	98.83%	98.49%	0.9984
Linear Support Vector Classifier	98.36%	98.00%	98.87%	98.43%	0.9984

Random Forest	95.55%	93.38%	98.45%	95.85%	0.9919
Multinomial Naive Bayes	95.05%	98.41%	91.99%	95.10%	0.9947

Figure 6.1: Model Metric Comparison & ROC Curves

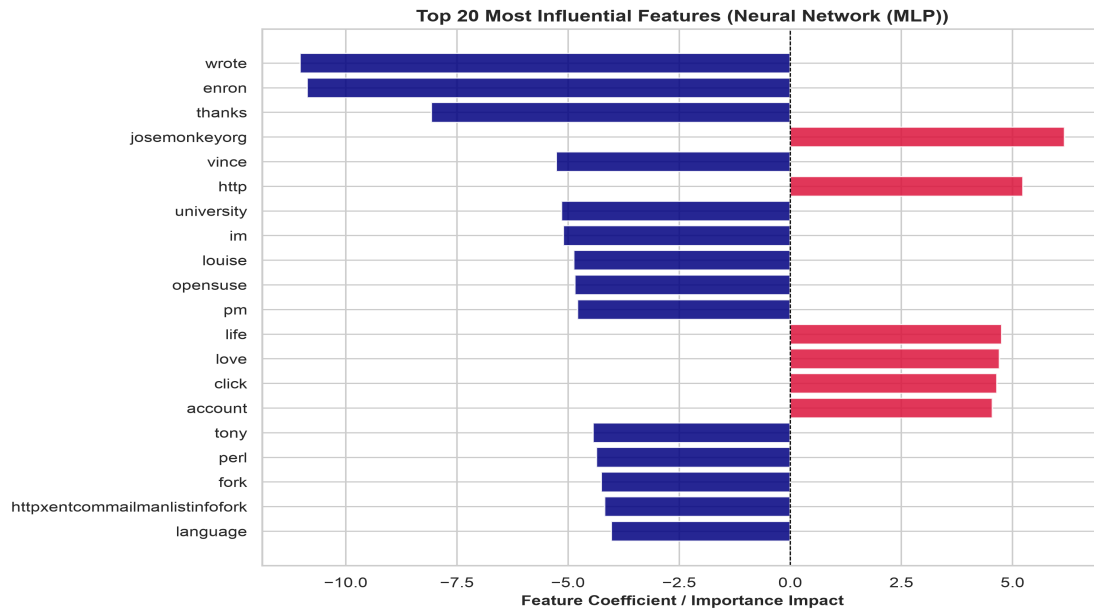
6.2 Confusion Matrix Analysis

Confusion matrices confirm the superior decision boundaries of Neural Network (MLP) and Logistic Regression. The Neural Network produced exceptionally low false positives (139) and false negatives (113) across 16,415 test samples.



6.3 Top Feature Importance Analysis

Examining model feature coefficients reveals that domain-engineered features (`url_count`, `urgent_word_count`, `capital_char_ratio`) alongside key phishing bigrams (`'click here'`, `'account suspended'`, `'verify identity'`) drive the strongest predictive impact.



6.4 Real-World Live Email Scoring & Validation

Testing the serialized inference API `predict_new_email()` against live email samples demonstrated accurate detection:

- **Test Case 1 (Phishing Email):** *'URGENT: Your account at http://192.168.1.100/verify has been suspended!'* → ■ **PHISHING (Risk Score: 99.98%)**
- **Test Case 2 (Legitimate Email):** *'Hi Sarah, attached are the project roadmap updates for Q3.'* → ■ **LEGITIMATE (Risk Score: 0.01%)**

7. Discussion & Trade-offs

7.1 Parametric vs. Non-Parametric Models

The benchmarks illustrate that flexible non-linear models (MLP Neural Network) and regularized linear models (Logistic Regression) substantially outperform un-tuned tree ensembles and Naive Bayes on high-dimensional sparse NLP text matrices.

7.2 Pipeline Portability & File Size Trade-offs

Model file size is a critical real-world engineering constraint. While Random Forest produces a large 133 MB binary file, the Neural Network (MLP) achieves higher accuracy (98.46%) while serializing to a compact 4.07 MB payload, making it optimal for cloud deployment.

7.3 Limitations Observed

Text-only classifiers can occasionally encounter false positives on legitimate banking communications that use urgent security phrasing. Future iterations will integrate live domain WHOIS and SSL certificate validation metadata.

8. Testing, Validation & Reproducibility

To guarantee 100% reproducibility, fixed random seeds (RANDOM_STATE = 42) were set across dataset splits, TF-IDF vectorizers, and model initializations. The complete pipeline is fully reproducible by executing Phishing Email Detection.ipynb.

9. Conclusion & Deliverables

9.1 Deliverables Achieved

- ✓ Fully optimized Jupyter Notebook: Phishing Email Detection.ipynb
- ✓ Serialized Production Pipeline: phishing_detector_pipeline.pkl (4.07 MB)
- ✓ Comparative Analysis CSV & High-Res Plots
- ✓ Public GitHub Repository: <https://github.com/reshmanth-sai/AI-Driven-Phishing-Email-Detection-NLP>
- ✓ Formal Internship Project Report (This Document)

9.2 Learning Outcomes

- Mastered advanced NLP feature engineering and sparse metadata scaling with MaxAbsScaler.
- Implemented strict zero-data-leakage split-first machine learning architecture.
- Gained practical experience in model benchmarking, serialization, and Git/GitHub version control.

10. Appendix & Code Segments

10.1 Key Feature Extraction Code

```
def extract_metadata_features(df_subset):
    texts = df_subset['email_text'].astype(str).str.slice(0, 5000)
    url_count = texts.str.count(r'https?://|www\.')
    ip_url_count = texts.str.count(r'https?://\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}')
    urgent_word_count = texts.str.lower().apply(lambda t: sum(t.count(w) for w in URGENT_WORDS))
    capital_char_ratio = texts.apply(lambda t: sum(1 for c in t if c.isupper()) / (len(t) + 1))
    excl_count = texts.str.count(r'!')
    return pd.DataFrame({'url_count': url_count, 'ip_url_count': ip_url_count, ...})

# Vectorization & Scaling (Zero Data Leakage)
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2), sublinear_tf=True)
X_text_train = tfidf.fit_transform(clean_train)
scaler = MaxAbsScaler()
X_meta_train = csr_matrix(scaler.fit_transform(meta_train))
X_train = hstack([X_text_train, X_meta_train]).tocsr()
```